CLAUDE CODE WORKSHOP · ANTHROPIC

Tokens & cache : le mémo d'une session

1 page
source : « Maximizing the value of Claude Code » · 26 min

« Efficient doesn't mean fewer tokens. It means the ones you use go toward what you actually asked for. » — logique du préfixe, hygiène de contexte, effort et modèle.

1 UN PROMPT N'EST JAMAIS UNE REQUÊTE



contexte qui grandit

préfixe caché (état serveur) · input nouveau · résultat d'outil · output (tool calls, thinking)

Chaque envoi = system prompt + définitions d'outils + env/git + `CLAUDE.md` + noms de skills & MCP + **ton message**. Puis boucle : tool call → exécution → résultat renvoyé. **La barre grossit à chaque aller-retour et rien n'en sort jamais.**

INPUT	CACHE READ	CACHE WRITE 5 MIN	CACHE WRITE 1 H	OUTPUT
1×	**0,1×**	**1,25×**	**2×**	**le + cher**
parallélisable → bon marché	le levier n°1	payé une fois	si pauses longues	1 passe complète / token

4 LEVIERS

- **Tokens in** — ce qui est chargé, la durée de session
- **Tokens out** — l'effort avant de conclure
- **Part en cache** — préfixe stable, cache chaud
- **Modèle** — multiplicateur des trois autres

2 CHECKLIST DE SESSION

AVANT — CHOISIR, PUIS NE PLUS TOUCHER

- ☐ **Modèle au bon niveau** — Sonnet = généraliste, Opus = expert (bug subtil), Haiku = mécanique. Le modèle multiplie *tous* les autres coûts.
- ☐ `/effort` **par défaut** — medium suffit presque toujours (excellent sur Fable 5.1).
- ☐ `/context` — voir ce qui est chargé et couper l'inutile.
- ☐ `/mcp` — désactiver les serveurs inutilisés ; préférer un CLI (gh, aws).
- ☐ **Nettoyer** `CLAUDE.md` — que l'universel. Le spécifique → **skill** (chargée à la demande).

PENDANT — CE QUE TU METS DANS LA CONVERSATION

- ☐ `@fichier.ts` — injecté avant l'envoi : un aller-retour économisé.
- ☐ **Sortie de commande silencieuse** — les 400 lignes « OK » restent dans *toutes* les requêtes suivantes. `--reporter=dot`.
- ☐ **Commandes du quotidien dans** `CLAUDE.md` — écrites exactement comme tu les lances.
- ☐ **Fouille bruyante → subagent** — il lit 30 fichiers, ne rend que la conclusion.
- ☐ **Ne pas changer modèle / effort** — ça casse le préfixe (sauf l'effort sur Fable 5.1).

APRÈS — GARDER LA SESSION FOCALISÉE

- ☐ `/clear` **à chaque nouvelle tâche** — sinon tu paies la tâche 1 pendant la tâche 3.
- ☐ `/compact` ***avant*** **de t'éloigner** — pendant que le cache est chaud ; sinon la relecture est au prix plein.
- ☐ **Reprise le lendemain ?** — `/compact` la veille : `claude --resume` relit tout à froid.
- ☐ `/rewind` — jeter les derniers tours plutôt que tout compacter.

3 LES PIÈGES DE CACHE

ACTION	CACHE	CONSÉQUENCE
`/model`	CASSE	cache = état interne du modèle → relecture complète
`/effort`	CASSE	sauf **Fable 5.1** : l'effort est dans le préfixe
`/compact` · `/clear`	CASSE	inévitable : on réécrit le préfixe
`/resume` après pause	CASSE	TTL expiré → tout au prix plein, une fois
`/rewind`	CASSE	tronque le préfixe
Éditer un fichier	INTACT	côté Claude Code, jamais dans la requête
Mode de permissions	INTACT	hors requête
Éditer `CLAUDE.md`	INTACT	lu une fois au démarrage → effet au prochain `/clear`

> **Règle du préfixe :** le serveur ne réutilise que la partie **strictement identique depuis le premier token**. Un seul token différent → tout ce qui suit est recalculé au prix plein.

4 QUEL RÉGLAGE CORRIGER ?

LE MODÈLE
« Il ne savait pas. »
Résultat faux, concept mal compris, territoire inconnu.
→ **monter de modèle.**

L'EFFORT
« Il n'a pas assez essayé. »
Fichier sauté, test non lancé, abandon à moitié.
→ **monter l'effort, ou le baisser s'il en fait trop.**

1 **Choisir au début, ne plus toucher.** Modèle, effort et MCP au démarrage — ou juste après un `/clear`.

2 **Être délibéré sur le contexte.** `@` les fichiers connus, output silencieux, fouille en subagent.

3 **Garder la session focalisée.** Un `/clear` par tâche, un `/compact` avant la pause.

D'après le Claude Code Workshop « Maximizing the value of Claude Code » (Anthropic), ~26 min. TTL & tarifs susceptibles d'évoluer : vérifier la doc avant de s'y fier en production.